

AI Engineering

Agentic Dependency Parsing in Low-Resource Settings

Dang Nhat Quang, Han Dai, Nikhila Gadge, Jan Raring

Submission: 19 July 2026

Abstract

In natural language processing (NLP), dependency parsing refers to the task of constructing sentence-level dependency trees. Often, the resulting dependency–head relations are annotated to specify the type of dependency. Large collections of such dependency trees—so called *treebanks*—are a valuable resource in the study of syntax, grammar, and typology. The manual creation of such dependency trees, however, is a very laborious undertaking which should, if possible, be performed at least semi-automatically. For high-resource languages, many popular NLP libraries (Stanza, NLTK, SpaCy) provide the necessary tooling for exactly this sort of (pre-)annotation. In the case of low-resource languages, however, such tools are typically not available. This observation has instigated us to test whether an AI agent, i.e. an LLM equipped with adequate tools, can function as a general purpose dependency parser, thereby eliminating the need to train purpose-built neural networks as used by the aforementioned libraries.

1 Introduction

In natural language processing (NLP), dependency parsing refers to the task of constructing sentence-level dependency trees. Syntactically, a dependency tree is a directed, connected, acyclic graphs with words as nodes and *dependency-head*-relations as edges. Semantically, it reflects a sentence’s dependency structure: Each word is linked to the head of the phrase that it is part of. There is always exactly one word that does not itself have a head. This word is called the *root* of the sentence. For additional specification, the edges of such a dependency tree are often annotated with a label taken from a pre-defined set of possible dependency relations.

Large collections of such dependency trees—so called *treebanks*—are being used in a number of linguis-

tic subfields: In syntax they shed light on the possible sentence structures a language allows for; in grammar they help explain to what sort of agreement constraints words in a phrase are subjected to; and in linguistic typology, languages are classified and compared based on the prevailing patterns in word order.

One popular guideline for dependency annotation is the *Universal Dependencies* (UD) framework. Its main proposition lies in the specification of conventions and tag sets for the annotation of parts of speech, morphological features, and dependency relations that are universal enough to be applicable to virtually all natural languages. Currently, the official UD collection contains 353 *treebanks* for 193 different languages. UD also defines the CoNLL-U file format used for storing

annotated sentences and entire treebanks along with metadata in a machine-readable manner. In this format, sentences are represented as tables with one row for every token/word and ten columns holding the annotations: ID, FORM, LEMMA, UPOS, XPOS, FEATS, HEAD, DEPREL, DEPS, and MISC. For now it suffices to know that ID is an integer index starting at 1, and that HEAD stores the ID of the words head or 0 if it is the root. The type of dependency relation can be specified in DEPREL and DEPS.

The creation of a treebank like this typically requires a lot of manual labor. To reduce the work load as much as possible, many projects opt for a semi-automated approach, using a pre-trained tagger for intermediate annotations that, in a second step, get corrected by a human. For high-resource languages, taggers are available via all major NLP libraries (Stanza, SpaCy, NLTK), but for the majority of the worlds roughly 7000 languages no such tools exist.

As the training of a tagger itself postulates the existence of a treebank to serve as training data, the problem of not having a pre-trained model is not one to be resolved easily. Additional challenges stem from the fact that many low resource languages have not undergone standardization and so the little amount of data that might be available displays high entropy, causing and training signal to be quite noisy. This throws annotators back into the situation of having to do everything by hand, which—depending on the availability of funding—can make the creation of a new treebank simply unaffordable.

But there is hope: The success of large language models (LLMs) as universal problem solvers gives us good reason to believe that the aforementioned bottle neck relating to manual labeling is indeed surmountable. More specifically, hundreds of treebanks can be assumed to be part of the data used for model training and even if a particular low-resource language is largely out of distribution, studies on cross-lingual transfer have shown that LLMs are reasonably good at extrap-

olating knowledge of a higher-resource language to a closely related low-resource language.

What makes dependency parsing nonetheless challenging is the topology of the problem: Dependency trees are inherently non-linear and dependency relations can stretch over long distances—conditions that LLMs are known to be less good at. The aim of this project is to measure the performance of smaller LLMs in the 7B–12B range on this task and to see if providing the model with different tools for example-retrieval, morphological analysis and lexicon look-ups has an impact on the output quality.

2 Related Work

Dependency parsing: Stanza, SpaCy, NLTK.

Cross-lingual transfer: (García-Ferrero)

3 Our Approach

4 Experiment

5 Results

6 Conclusion

7 Author Contribution Statement

Everyone explains the extend of their contribution in their own words.

Dang Nhat Quang. Expand...

Han Dai

Nikhila Gadge

Jan Raring

Bibliography

García-Ferrero, Iker. “Cross-Lingual Transfer for Low-Resource Natural Language Processing.” 2025, <https://arxiv.org/abs/2502.02722>.